

GRUPA 5/5 — na svakom roku

1. Otpornost na greške — maksimalan broj procesa koji sme da otkáže (jun 2024)

Zadatak: U grupi postoji deset repliciranih procesa. Ako mogu nastupiti samo mirne greške, koliko maksimalno procesa može otkazati a da se ipak dobije korektan rezultat? Ako mogu nastupiti greške vizantijskog tipa, koliko procesa može maksimalno otkazati a da se ipak dobije korektan rezultat? Šta ako mogu nastupiti greške vizantijskog tipa a procesi moraju postići konsenzus? Koliko u ovom slučaju maksimalno procesa može otkazati? Svaki odgovor obrazložiti.

mirne greske:	potrebno	$k + 1$	procesa
Vizantijske greske:	potrebno	$2k + 1$	procesa
Vizantijske + konsenzus:	potrebno	$3k + 1$	procesa

$n = 10$:

a) mirne: $k + 1 \leq 10 \Rightarrow k = 9$

b) Vizantijske: $2k + 1 \leq 10 \Rightarrow k \leq 4.5 \Rightarrow k = 4$ (ceo broj, nadole!)

c) konsenzus: $3k + 1 \leq 10 \Rightarrow k = 3$

a) $k = 9$ — mirna greška znači da proces prestane da radi i ćuti (ne šalje pogrešne rezultate), pa je dovoljno da ostane bar 1 ispravan proces koji daje rezultat. b) $k = 4$ — vizantijski procesi rade i lažu, pa mora većinsko glasanje: ostaje 6 ispravnih koji nadglasaju 4 lažova ($6 > 4$). c) $k = 3$ — za konsenzus (Lamportov algoritam) mora biti bar $2k+1 = 7$ lojalnih, tj. više od $2/3$ ukupnog broja ($7/10 > 2/3$).

Varijanta $n = 5$ (BELO ZLATO, pitanje 2): mirne 4, vizantijske 2 (3 ispravna nadglasaju 2), konsenzus 1 ($3 \cdot 1 + 1 = 4 \leq 5$; ostaje 4 lojalna, $4/5 = 80\% > 2/3$; za $k = 2$ trebalo bi $7 > 5$ procesa, pa $k = 2$ nije moguće). PAŽNJA: u skripti BELO ZLATO kod ovog pitanja postoji greška — ispravne formule su gornje.

RECEPT: 1-2-3: ćute $\rightarrow 1 \cdot k + 1$; lažu $\rightarrow 2 \cdot k + 1$; lažu i moraju da se slože $\rightarrow 3 \cdot k + 1$. Reši nejednačinu "potrebno $\leq n$ ", zaokruži NADOLE. Obrnuta formulacija ("koliko replika za f grešaka"): $f+1$, $2f+1$, $3f+1$.

2. ABFT — detekcija i korekcija greške u matrici (okt2 2025)

Zadatak: U procesu množenja dve matrice korišćena je ABFT tehnika za detekciju i korekciju grešaka. Dobijen je sledeći rezultat (poslednja kolona = kontrolne sume vrsta, poslednja vrsta = kontrolne sume kolona): a) Utvrditi da li je nastupila greška u toku izračunavanja; b) Ako je nastupila greška, naći njenu lokaciju; c) Izvršiti korekciju greške, ako je greška nastupila.

	1	2	3		6		ABFT = informaciona redundansa
	2	3	4		9		
	3	3	3		8		
	6	7	10		23		

a) Vrste: $1+2+3=6$ ☐ $2+3+4=9$ ☐ $3+3+3=9 \neq 8$ ☐
Kolone: $1+2+3=6$ ☐ $2+3+3=8 \neq 7$ ☐ $3+4+3=10$ ☐
=> greska JESTE nastupila.

b) Presek 3. vrste i 2. kolone => element C32 = 3.

c) nova = stara + (kontrolna - stvarna)
po vrsti: $C32 = 3 + (8 - 9) = 2$
po koloni: $C32 = 3 + (7 - 8) = 2$ (poklapaju se)
Provera: $v3: 3+2+3 = 8$ ☐ $k2: 2+3+2 = 7$ ☐ $ugao: 6+9+8 = 23 = 6+7+10$ ☐

RECEPT: Saberi SVE vrste i SVE kolone. Tačno 1 vrsta + 1 kolona ne štimaju → greška u podatku na preseku; korekcija = stara + (kontrolna - stvarna), pa provera. Samo vrsta (ili samo kolona) ne štima → pogrešna je sama kontrolna suma. Sve štima → greška NIJE nastupila. Dve greške: detekcija da, jednoznačna korekcija ne.

3. Konzistencija — METOD (važi za zadatke 4–10)

Pitanje svakog zadatka: da li je ovo moglo da se desi na JEDNOM računaru sa jednom kopijom podataka? Traži se jedan globalni redosled svih operacija u kome: 1) svaki proces zadržava svoj programski red; 2) svako čitanje vraća vrednost poslednjeg upisa pre njega.

Strelica " $A < B$ " = A mora biti pre B. Izvori strelica:

1) PROGRAMSKI RED – operacije procesa redom kako pisu u zadatku
(NIKAD se ne smeju premestati!)

2) VREDNOST CITANJA:

R je dobio STARU vrednost => $R < W$ (citanje pre upisa)

R je dobio NOVU vrednost => $W < R$ (upis pre citanja)

vise upisa u istu promenljivu => citanje vraća POSLEDNJI upis pre njega

Nadovezi strelice:

NIZ $A < B < C < D$ (svako ima mesto) => MOGUĆE – ispisi taj niz

KRUG $A < B < C < A$ (A pre samog sebe) => NIJE MOGUĆE – pokazi krug

RECEPT: Za "MOGUĆE" dokaz je ISPISAN redosled + provera čitanja. Za "NIJE MOGUĆE" dokaz je ciklus. Bez zaključka (moguće/nije) zadatak se ne bodeje.

4. Sekvencijalna konzistencija — dve replike (jun 2024)

Zadatak: Tri procesa P1, P2 i P3 izvršavaju instrukcije nad tri deljive promenljive x, y i z (P1: x=1; print(y,z) · P2: y=1; print(x,z) · P3: z=1; print(x,y)), inicijalizovane na nulu. Postoje dve replike R1 i R2. Operacije na replikama se izvode u sledećem redosledu — R1: x=1, print(y,z), y=1, print(x,z), z=1, print(x,y) · R2: x=1, y=1, print(x,z), print(y,z), z=1, print(x,y). Da li je ovakvo skladište podataka sekvencijalno konzistentno? Obrazložiti odgovor.

NIJE sekvencijalno konzistentno. Sekvencijalna konzistencija zahteva da SVE replike vide JEDAN isti redosled svih operacija. U R1 se print(y,z) izvršava PRE y=1 (ispisuje y=0), a u R2 POSLE y=1 (ispisuje y=1) — replike ne vide isti redosled, pa jedinstveni sekvencijalni redosled ne postoji. Programski red jeste ispoštovan u obe replike, ali to nije dovoljno.

5. Sekvencijalna konzistencija — mogući ispisi (okt 2025 / SP)

Zadatak: Tri konkurentna procesa (P1: x=1; y=1 · P2: z=y · P3: print(x,y,z)); x, y, z inicijalno 0. Za svaki od navedenih rezultata štampanja odgovoriti i objasniti da li je moguć pod uslovom da je skladište podataka sekvencijalno konzistentno: a) x=0, y=1, z=1; b) x=1, y=1, z=0; c) x=1, y=0, z=1.

- a) x=0,y=1,z=1 NIJE MOGUĆE: x=0 => print pre x=1; y=1 => y=1 pre print;
prog. red P1: x=1 pre y=1 => x=1 < y=1 < print < x=1 — KRUG.
- b) x=1,y=1,z=0 MOGUĆE: redosled z=y (z=0), x=1, y=1, print(1,1,0) □
- c) x=1,y=0,z=1 NIJE MOGUĆE: z=1 => y=1 pre z=y pre print; a y=0 na
ispisu => print pre y=1 — KRUG.

6. Model konzistencije koji dozvoljava date ispise (okt 2025 / SP)

Zadatak: Dva konkurentna procesa (P1: x=1; print(x,y) · P2: y=1; print(x,y)); x i y inicijalno 0. Da li postoji model konzistencije kod koga je moguće da proces P1 štampa x=1, y=0, a proces P2 x=0, y=1? Obrazložiti odgovor.

DA — uslovna (kauzalna) konzistencija (i svaki slabiji model, npr. FIFO). Upisi W(x)1 i W(y)1 su konkurentni — nijedan proces nije pročitao vrednost drugog pre svog upisa, pa nema potencijalne uslovljenosti, a kauzalna konzistencija dozvoljava da se konkurentni upisi vide u različitom redosledu u različitim procesima. Kod sekvencijalne NIJE moguće: zahtevalo bi $W(x) < P1.print < W(y) < P2.print < W(x)$ — ciklus.

7. Sekvencijalna konzistencija — tri procesa, izlazi 10 / 11 / 01 (okt2 2025)

Zadatak: Tri konkurentna procesa (P1: x=1; print(y,z) · P2: y=1; print(x,z) · P3: z=1; print(x,y)); x, y, z inicijalno 0. Ako proces P1 generiše izlaz 10, proces P2 izlaz 11, a proces P3 izlaz 01, da li je dobijeni rezultat korektan sa stanovišta sekvencijalne konzistencije? Objasniti zašto.

Ispisi: P1 -> y=1,z=0; P2 -> x=1,z=1; P3 -> x=0,y=1.
P3 vidi x=0 => P3.print < x=1
prog. red P3: z=1 < P3.print
prog. red P1: x=1 < P1.print
=> z=1 < P3.print < x=1 < P1.print
P1 vidi z=0 => P1.print < z=1 — KRUG!

NIJE korektno — lanac se zatvara u ciklus (z=1 bi morao biti pre samog sebe), pa ne postoji jedinstveni sekvencijalni redosled.

8. Sekvencijalna konzistencija — oba procesa čitaju staru vrednost (jun 2026)

Zadatak: Inicijalne vrednosti deljivih promenljivih u repliciranom skladištu podataka su $x = y = 0$. Aktivnosti procesa P1 i P2 su: P1: $W(x)=1, R(y)=0$ · P2: $W(y)=1, R(x)=0$. Da li je ovakvo ponašanje dozvoljeno u slučaju sekvencijalne konzistencije? Objasniti.

```
prog. red:  W(x)=1 < R(y)=0          W(y)=1 < R(x)=0
citanja:   R(y)=0 (stara) => R(y)=0 < W(y)=1
           R(x)=0 (stara) => R(x)=0 < W(x)=1
lanac:     W(x)=1 < R(y)=0 < W(y)=1 < R(x)=0 < W(x)=1  - KRUG!
```

NIJE dozvoljeno — $W(x)=1$ bi morao biti pre samog sebe. Intuitivno: oba procesa tvrde da nisu videli tuđ upis, tj. oba bi morala biti prva — na jednom računaru bar jedan mora videti upis onog drugog.

9. Koji model dozvoljava različit redosled čitanja dva upisa (jun 2026)

Zadatak: Inicijalno $x = 0$. Proces P1 i P2 se izvršavaju konkurentno: P1: $W(x)=1$ · P2: $W(x)=2$. Kasnije procesi P3 i P4 čitaju x : P3 reads: 1 zatim 2 · P4 reads: 2 zatim 1. Koji model konzistencije ovo dozvoljava?

Uslovna (kauzalna) konzistencija (i FIFO kao slabiji model). $W(x)_1$ i $W(x)_2$ su konkurentni upisi (iz različitih procesa, bez međusobnog čitanja), pa se po kauzalnoj konzistenciji mogu videti u različitom redosledu u različitim procesima. Sekvencijalna NE dozvoljava — svi procesi bi morali videti isti redosled, a P3 vidi $1 \rightarrow 2$, P4 vidi $2 \rightarrow 1$. Striktna NE dozvoljava.

10. Kauzalna konzistencija — lanac uslovljenih upisa (jul 2026)

Zadatak: Da li je sledeće skladište podataka kauzalno konzistentno: P1: $W(x)=1$ · P2: $R(x)=1 \rightarrow W(y)=1$ · P3: $R(y)=1 \rightarrow W(z)=1$ · P4: $R(z)=1 \rightarrow R(x)=0$? Obrazložiti odgovor.

NIJE kauzalno konzistentno. Lanac potencijalne uslovljenosti (tranzitivan): $W(x)=1 \rightarrow P2 \text{ čita } x=1 \rightarrow W(y)=1 \rightarrow P3 \text{ čita } y=1 \rightarrow W(z)=1$. Dakle $W(x)=1$ kauzalno prethodi upisu $W(z)=1$. P4 je video $W(z)=1$ ($R(z)=1$), pa mora videti i sve upise koji mu kauzalno prethode — a čita $R(x)=0$, staru vrednost. Kauzalnost je narušena.

11. Sekvencijalna konzistencija — odrediti vrednosti čitanja (jul 2026)

Zadatak: Da li ovakvo skladište podataka može biti sekvencijalno konzistentno: P1: $W(x)=1 \rightarrow R(y)=?$ · P2: $W(y)=1 \rightarrow R(x)=?$ · P3: $R(x)=1 \rightarrow R(y)=0$? Ako može, koje vrednosti treba da vrate $R(y)=?$ i $R(x)=?$ Ako ne može, pokazati zašto ne može.

```
P3 cita x=1 => W(x)=1 < P3.R(x);   P3 cita y=0 => P3.R(y) < W(y)=1
prog. red P2: W(y)=1 < P2.R(x)
lanac: W(x)=1 < P3.R(x) < P3.R(y) < W(y)=1 < P2.R(x)  =>  R(x) MORA biti 1
```

validan redosled: $W(x)=1, R(x)=1(P3), R(y)=0(P3), R(y)=0(P1), W(y)=1, R(x)=1(P2)$
MOŽE. Vrednosti: $R(y) = 0$ (u P1) i $R(x) = 1$ (u P2). $R(x)=1$ je prinudno (lanac gore); $R(y)$ u P1 može biti i 0 i 1 — obe vrednosti daju validan redosled, navodi se $R(y)=0$ uz gornji niz kao dokaz.

12. Sun RPC — pisanje definicije interfejsa (3 ispitne varijante)

Zadatak: VARIJANTA A (jul 2026): Napisati Sun RPC definiciju interfejsa koji omogućava pristup jednoj udaljenoj proceduri koja na osnovu skupa izmerenih temperatura pronalazi minimalnu i maksimalnu izmerenu temperaturu. Napisati kako bi izgledao poziv udaljene procedure u klijentskoj aplikaciji. Navesti potpis odgovarajuće procedure u serverskoj aplikaciji.

```
/* temp.x */
const MAXTEMP = 100;
struct temp_data { float temps<MAXTEMP>; }; /* niz promenljive velicine */
struct min_max { float min; float max; }; /* 2 rezultata => struct */

program TEMP_PROG {
    version TEMP_VERS {
        min_max FIND_MIN_MAX(temp_data) = 1; /* broj procedure */
    } = 1; /* broj verzije */
} = 0x20345678; /* broj programa */

/* klijent: */
CLIENT *cln = clnt_create(server, TEMP_PROG, TEMP_VERS, "udp");
td.temps.temps_len = n; td.temps.temps_val = niz;
min_max *rez = find_min_max_1(&td, cln); /* malim slovima + _1 */
printf("min=%f max=%f\n", rez->min, rez->max);

/* potpis na serveru (rezultat MORA static — vraća se pokazivac): */
min_max *find_min_max_1_svc(temp_data *argp, struct svc_req *rqstp);
```

Zadatak: VARIJANTA B (okt2 2025): procedura prima podatke o transakciji — identifikator (ceo broj) i iznos (realan broj, valuta USD) — a vraća niz realnih brojeva: originalni iznos, iznos u evrima i iznos u funtama. Prikazati poziv funkcije u klijentskoj aplikaciji.

```
/* okt2.x */
struct transakcija { int id; float iznos; };
struct iznosi { float vrednosti<3>; }; /* original, EUR, GBP */

program KONV_PROG {
    version KONV_VERS {
        iznosi KONVERTUJ(transakcija) = 1;
    } = 1;
} = 0x20111111;

/* klijent: */
transakcija t; t.id = 1; t.iznos = 100.0;
iznosi *rez = konvertuj_1(&t, cln);
printf("%f %f %f\n", rez->vrednosti.vrednosti_val[0],
        rez->vrednosti.vrednosti_val[1], rez->vrednosti.vrednosti_val[2]);
```

Zadatak: VARIJANTA C (jun 2026): sistem elektronskog glasanja — metoda vote (jedinstveni broj birača, identifikator kandidata, vreme glasanja; server proverava da li je birač već glasao i vraća informaciju o uspešnosti) i metoda getTopCandidates (lista identifikatora kandidata sortirana opadajuće prema broju osvojenih glasova).

```
/* glasanje.x */
struct glas { int biracId; int kandidatId; int vreme; };
struct lista { int kandidati<100>; };

program GLASANJE_PROG {
    version GLASANJE_VERS {
        int VOTE(glas) = 1; /* 1 = uspesno, 0 = vec glasao */
        lista GET_TOP_CANDIDATES(void) = 2; /* bez argumenta => void */
    } = 1;
}
```

```
} = 0x20222222;
```

RECEPT: Uvek: 1 procedura = 1 argument + 1 rezultat (više vrednosti → struct; lista → niz promenljive veličine <N>). Imena u .x VELIKIM slovima; klijent zove malim + _1; server implementira _1_svc sa static rezultatom; broj programa 0x2xxxxxxx.

GRUPA 4/5

13. Lamportove markice — da li markice dokazuju uzročnost (jul 2026)

Zadatak: U distribuiranoj bazi podataka koriste se Lamportove markice za uređivanje događaja. Dva ažuriranja dobijaju sledeće markice: Update X = 25 i Update Y = 27. Da li se može zaključiti da je ažuriranje X uzrokovalo ažuriranje Y? Obrazložiti odgovor.

NE, ne može se zaključiti. Kod Lamportovih markica važi samo jedan smer: ako je X uzrokovao Y, onda je $T(X) < T(Y)$ — ali obrnuto ne važi. Iz $25 < 27$ ne sledi uzročnost; X i Y mogu biti potpuno konkurentni (u procesima koji ne razmenjuju poruke). Jedino što se sme zaključiti: Y sigurno NIJE uzrokovao X. Za utvrđivanje uzročnosti potrebni su vektorski časovnici.

14. Vektorski časovnici — koji vektor nije moguć (jul 2026)

Zadatak: Događaji A, B, C i D u distribuiranom sistemu imaju sledeće vektorske časovnike: A [1,0,0], B [2,0,0], C [2,1,0], D [1,2,0]. Koji od gore navedenih vektorskih časovnika nije moguć i zašto?

A[1,0,0] -> 1. događaj procesa P1
B[2,0,0] -> 2. događaj procesa P1
C[2,1,0] -> 1. događaj procesa P2 (zna za 2 događaja iz P1)
D[1,2,0] -> 2. događaj procesa P2 (zna za SAMO 1 događaj iz P1?!)

Nije moguć D [1,2,0]. C i D su prvi i drugi događaj procesa P2 (druga komponenta 1 pa 2), a unutar istog procesa komponente vektorskog časovnika ne mogu da opadnu — prva komponenta je pala sa 2 (u C) na 1 (u D), tj. P2 bi "zaboravio" da se u P1 desio drugi događaj. Korektna vrednost bila bi D [2,2,0].

RECEPT: Grupiši događaje po procesima (komponenta koja raste = vlasnik događaja), pa u svakoj grupi proveriti da nijedna komponenta ne opada. Prekršilac je odgovor + napiši korektnu vrednost.

15. Vektorski časovnici — uslovljeni i konkurentni događaji sa slike (okt2 2025)

Zadatak: Na slici su prikazana tri procesa koja međusobno komuniciraju. Korišćenjem vektorskih časovnika odrediti koji su događaji međusobno uslovljeni a koji konkurentni. (Zadatak dolazi sa slikom — postupak važi za svaku sliku.)

Pravila dodele vektora (N = broj procesa):

- 1) svi vektori kreću od [0,...,0]
- 2) P_i pre SVAKOG svog događaja inkrementira svoju komponentu: $V_i[i]++$
- 3) poruka nosi vektor posiljaoca; prijemnik za svaku komponentu uzima $\max(\text{svoj}, \text{primljeni})$, pa primeni pravilo 2

Poređenje dva događaja e i e':

$V(e) < V(e')$ (sve komponente $\leq$, bar jedna $<$) $\Rightarrow$ e -> e' USLOVLJENI
ne može ni $V(e) \leq V(e')$ ni $V(e) \geq V(e')$ $\Rightarrow$ KONKURENTNI

Na ispitu: obeleži svaki događaj vektorom po pravilima 1–3 prateći strelice poruka, pa uporedi tražene parove — par kod koga jedan vektor ima veću jednu, a manju drugu komponentu je konkurentan; par kod koga su sve komponente jednog $\leq$ drugog je uslovljen.

GRUPA 3/5

16. Broj poruka za kritičnu sekciju (jun 2026 / jul 2026 — isti zadatak)

Zadatak: Sistem ima 10 procesa. Koliko je poruka potrebno razmeniti da bi proces ušao i napustio kritičnu sekciju ako se koristi: a) Centralizovani algoritam; b) Ricart–Agrawala algoritam?

- a) Centralizovani: 3 poruke (REQUEST, OK, RELEASE)
– ne zavisi od broja procesa!
- b) Ricart-Agrawala: $2(n-1) = 2 * 9 = 18$ poruka
($n-1$ REQUEST svima ostalima + $n-1$ OK potvrda od svih)

Ako pitanje traži samo ulazak: centralizovani 2 (REQUEST + OK), uz RELEASE pri napuštanju ukupno 3. Kod R-A napuštanje ne dodaje poruke — OK poruke koje proces pri izlasku šalje čekaocima pripadaju NJIHOVIM ciklusima ulaska. Zamka: $2(n-1)$, ne $2n$ — sebi se ne šalje!

17. Kvorum — izbor Read i Write kvoruma (jul 2026)

Zadatak: Distribuirani sistem koristi protokol baziran na kvorumu. U sistemu postoji 12 replika. Koje vrednosti Read i Write kvoruma je najbolje odabrati ako se čitanja obavljaju često a ažuriranja retko? Obrazložiti.

Uslovi (Gifford): 1) $N_R + N_W > N$ sprečava READ-WRITE konflikte
(read i write kvorum se uvek preklapaju, pa čitanje zakaci bar jednu svezu kopiju)

2) $N_W > N / 2$ sprečava WRITE-WRITE konflikte
(dva istovremena upisa moraju deliti server)

Citanja cesta => minimalan N_R :

$$N_R = 1 \Rightarrow 1 + N_W > 12 \Rightarrow N_W = 12 \quad \text{provera: } 12 > 6 \quad \square$$

$N_R = 1, N_W = 12$. Čitanje kontaktira samo jedan server pa su česta čitanja jeftina; skupo ažuriranje svih 12 replika plaća se retko. Obrnut slučaj (česti upisi): $N_W = 7$ (minimum $> N/2$), $N_R = 6$. Varijanta $N = 20$ (BELO ZLATO 50): dato $N_R = 1 \rightarrow N_W = 20$; dato $N_W = 11 \rightarrow N_R + 11 > 20 \rightarrow N_R = 10$. Kad je jedan kvorum DAT, samo ubaci u nejednačinu — ne pravi tabelu kombinacija.

18. HDFS — broj i veličina blokova (jun 2024)

Zadatak: Pretpostavimo da je fajl veličine 514MB sačuvan na HDFS-u. Ako je veličina bloka 64MB i podrazumevani faktor replikacije 4, koliki je ukupni broj blokova i kolika je veličina svakog od njih?

$514 / 64 = 8$ punih blokova + ostatak 2MB => 9 blokova po kopiji
(poslednji blok zauzima SAMO koliko nosi — 2MB, ne 64MB!)

ukupno: $9 \times 4 = 36$ blokova
od toga: $8 \times 4 = 32$ bloka po 64MB
 $1 \times 4 = 4$ bloka po 2MB
zauzeće: $514\text{MB} \times 4 = 2056\text{MB}$

RECEPT: $\square \text{fajl} \div \text{blok} \square \text{ blokova po kopiji (puni + ostatak)} \rightarrow \times \text{ faktor replikacije} \rightarrow \text{pobroji obe veličine.}$
Zauzeće = veličina fajla $\times$ faktor replikacije (nikad broj_blokova $\times$ pun_blok!).

19. NTP — na koju vrednost se postavlja klijentski časovnik (okt 2025 / SP)

Zadatak: Časovnik na računaru se sinhronizuje korišćenjem NTP protokola. Zabeležena su sledeća vremena: a. vreme na klijentu kada upućuje zahtev serveru: 6:22:15.100; b. vreme prijema zahteva na serveru: 7:05:10.700; c. vreme kada server šalje odgovor: 7:05:10.710; d. vreme na klijentu kada prima odgovor od servera: 6:22:15.250. Na koju vrednost će se postaviti klijentski časovnik?

$t_0 = 6:22:15.100$ (klijent salje) $t_1 = 7:05:10.700$ (server prima)
 $t_2 = 7:05:10.710$ (server salje) $t_3 = 6:22:15.250$ (klijent prima)

ceo krug (klijentov sat): $t_3 - t_0 = 150$ ms
obrada na serveru: $t_2 - t_1 = 10$ ms
mrežno kasnjenje (krug): $\text{delta} = 150 - 10 = 140$ ms $\Rightarrow$ jedan smer 70 ms

$t_3' = t_2 + \text{delta}/2 = 7:05:10.710 + 0.070 = 7:05:10.780$

(formule: $\text{delta} = (t_1 - t_0) + (t_3 - t_2)$; $\text{theta} = [(t_1 - t_0) + (t_2 - t_3)]/2$; $t_3' = t_3 + \text{theta}$)

Klijentski časovnik se postavlja na 7:05:10.780. Napomena: dobijeno vreme je veće od trenutnog pa se sat postavlja direktno; da je manje, korekcija bi se vršila postepeno (usporavanjem časovnika) — vreme ne sme ići unazad. Pretpostavka: kašnjenja zahteva i odgovora su približno jednaka (zato deljenje sa 2).

RECEPT: Računaj razlikama ISTOG sata: $(t_3 - t_0)$ i $(t_2 - t_1)$ — male cifre, bez mešanja satova. δ = krug – obrada; tačno vreme = $t_2 + \delta/2$.

20. Primary-backup protokol — koraci za write (jul 2026)

Zadatak: Distribuirani sistem koristi primary-backup protokol. Postoje 4 replike: R1, R2, R3, R4. R1 je primar. Klijent šalje write($x=10$). Redom navesti korake koji se dešavaju.

W1: klijent $\rightarrow$ lokalni server (npr. R2): write($x=10$)
W2: R2 $\rightarrow$ R1 (primar): prosledjuje zahtev — upisi idu SAMO kroz primar
W3: R1 izvrši upis $x=10$, pa šalje azuriranje SVIM replikama (R2, R3, R4)
W4: R2, R3, R4 $\rightarrow$ R1: potvrde (ACK) da su azurirale
W5: R1 $\rightarrow$ R2 $\rightarrow$ klijent: tek SADA klijent dobija potvrdu i nastavlja

Klijent je blokiran dok SVE replike ne potvrde (W5 tek posle svih W4) — zato posle upisa svako čitanje na bilo kojoj replici vidi novu vrednost. Primar uređuje sve upise u jedan globalni redosled, pa protokol postiže sekvencijalnu konzistenciju. Čitanja se obavljaju lokalno.

21. Ricart–Agrawala — baferi i ko ulazi prvi (jun 2026)

Zadatak: Pet procesa jednovremeno traži zahtev da pristupe istoj kritičnoj sekciji. Lamportove markice zahteva su P1: 9, P2: 6, P3: 6, P4: 12, P5: 10. Prikazati kako izgledaju baferi svakog procesa ako se koristi Ricart-Agrawala algoritam. Koji zahtevi će odmah biti potvrđeni?

Remi P2/P3 (obe markice 6): pobedjuje MANJI id => P2 pre P3.

Prioritet: P2(6,2) < P3(6,3) < P1(9) < P5(10) < P4(12)

Bafer = zahtevi LOSIJIH od mene (njima ne odgovaram); boljima saljem OK.

P2: bafer = P3, P1, P5, P4 (OK ne salje nikome)

P3: bafer = P1, P5, P4 (OK -> P2)

P1: bafer = P5, P4 (OK -> P2, P3)

P5: bafer = P4 (OK -> P2, P3, P1)

P4: bafer prazan (OK -> svima)

Odmah se potvrđuje samo zahtev procesa P2 — jedini dobija sva 4 OK i ulazi u kritičnu sekciju. Kada P2 izađe, šalje OK svima iz svog bafera → ulazi P3, pa P1, pa P5, pa P4 (redosled ulaska = redosled prioriteta). Baferi prave "stepenice" 4-3-2-1-0 — ako ne ispadnu, negde je greška.

GRUPE 2/5 i 1/5

22. Chord — pridruživanje čvora prstenu (okt 2025 / SP)

Zadatok: U Chord prstenu koji koristi 5-to bitne identifikatore prisutni su čvorovi sa identifikatorima 1, 4, 9, 11, 14, 18, 20, 21, 28. Pretpostavimo da se čvor sa identifikatorom 15 pridružuje Chord prstenu tako što kontaktira čvor sa identifikatorom 20. Redom napisati korake koji opisuju pridruživanje čvora 15 prstenu.

1. N15 kontaktira N20 i poziva `join(N20)`
2. N20 pronalazi neposrednog sledbenika za N15 -> to je N18
(prvi aktivni cvor sa ID >= 15; ostali cvorovi se NE obavestavaju)
3. N15 postavlja N18 za sledbenika i javlja N18: "ja sam tvoj novi prethodnik"
4. N18 postavlja N15 za svog prethodnika
5. kljucevi iz opsega (14, 15] koji su bili kod N18 prebacuju se na N15
6. prethodnik se dodaje TEK KASNIJE — N14 periodično pokreće protokol stabilizacije: pita svog sledbenika (jos uvek N18) "ko ti je prethodnik?"
7. N18 odgovara "N15" => N14 postavlja N15 za sledbenika i javlja mu se
8. N15 postavlja N14 za svog prethodnika

Ključna rečenica: svaki čvor periodično izvršava protokol stabilizacije da bi otkrio novopridružene čvorove. Nevalidne finger tabele samo usporavaju pretragu — pronalaženje je i dalje moguće preko sledbenika/prethodnika.

23. Chord — primar, replike i otkaz čvora (jun 2026)

Zadatok: U Chord prstenu koji koristi 6-to bitne identifikatore prisutni su čvorovi sa identifikatorima 4, 12, 20, 35, 50. Faktor replikacije je 3. Ubačen je fajl sa identifikatorom 18. Ko je primarni vlasnik fajla sa identifikatorom 18? U kojim čvorovima se nalaze replike? Šta se dešava ako čvor 20 otkaže? Ko je novi primar za fajl sa identifikatorom 18? Gde se sada nalaze kopije tog fajla?

pravilo: fajl k pripada $\text{succ}(k)$ = prvi cvor sa ID >= k (u smeru kazaljke)

primar za 18: $\text{succ}(18) = 20$

replike (r=3): primar + 2 naredna sledbenika = 20, 35, 50

otkaz cvora 20:

- prethodnik (12) preko heartbeat-a ("Are you alive?") detektuje otkaz i preusmerava pokazivac na prvog zivog iz liste sledbenika -> 35
- kljucevi iz (12, 20], ukljucujuci fajl 18, prelaze sledbeniku -> 35
- novi primar: $\text{succ}(18) = 35$
- re-replikacija vraca faktor 3: kopije su sada 35, 50, 4
(prsten se zatvara — posle 50 sledi 4)

24. Bully algoritam — poruke do izbora novog koordinatora (jun 2026)

Zadatak: U distribuiranom sistemu sa 5 procesa, P1 P2 P3 P4 P5, za izbor koordinatora koristi se Bully algoritam. P5 je koordinator. Proces P2 detektuje otkaz koordinatora. Prikazati sve poruke koje će se razmeniti do izbora novog koordinatora.

1. P2 -> P3, P4, P5 : ELECTION (salje SVIMA sa vecim id-em; i mrtvom P5!)
2. P3 -> P2 : OK (ziv je => P2 ispada iz izbora)
P4 -> P2 : OK (P5 cuti — mrtav)
3. P3 -> P4, P5 : ELECTION (P3 pokrece svoje izbore)
4. P4 -> P3 : OK (=> P3 ispada; P5 cuti)
5. P4 -> P5 : ELECTION (jedini veci od P4)
6. ...timeout, P5 ne odgovara... => P4 JE POBEDNIK
7. P4 -> P1, P2, P3 : COORDINATOR

Novi koordinator: P4 (najveci ZIVI id). Ukupno 12 poruka.

Pravila: ELECTION ide samo procesima sa VEĆIM id-em (P2 ne šalje P1); ko primi ELECTION odgovara OK i pokreće svoje izbore; ko ne dobije nijedan odgovor u roku — pobednik je i šalje COORDINATOR svima. Najčešća greška: preskakanje međuizbora (koraci 3–5).